

Data streaming system for VLC air and weather metrics

Optimization & deployment

For the course DLBDSEDE02– tutor: Dr. Sahar Qadan

STUDENT: TORDA BALÁZS, 92128244 (Data Science BSc)

Finalized for submission on 31 Dec 2025

Table of Contents

Abbreviations and definitions ii

An abstract finalization of the VLC pipeline 1

1. Object(ive)s 1

2. Breakdown of implementation 1

3. Result..... 2

4. Reflection 2

 Virtually freezing 2

 Virtually out of memory 2

BIBLIOGRAPHY 3

APPENDIX..... 4

Screenshots displaying milestones during finalization/optimization 4



Figure 1: The A03_MOLISOL_60m air quality station (photo by the author)

Abbreviations and definitions

Abbrev	Longform (= English)
(.)y(a)ml	Yet Another Markup Language
(pg)JDBC	(postgreSQL) Java Database Connectivity (driver)
.py	Python script
.sh	shell script
.txt	text()file
API	application programming interface
BSc	Bachelor of Science
CI/CD	continuous integration / continuous delivery/deployment
DB	database
Dec	December
DLBDSEDE02	Distance Learning Bachelor Data Science Elective Data Engineering – course #2
DR/Dr(.)	doctor
ext4	fourth extended filesystem
fiwareid	Future Internet WARE identifier
gh	github
GiB	gibibyte
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
JMX	Java Management Extensions
JVM	Java virtual machine
NO2	nitrogen dioxide
NSG	Network Security Group
O3	ozone
ODS	Opendatasoft
OOM	out of memory
P	phase
PK	primary key
pm	particulate matter
RAM	random-access memory
RSS	resident set size
SO2	sulphur dioxide
SQL	Structured Query Language
SSH/ssh(d)	Secure Shell (daemon)
timestamptz	Timestamp with Time Zone
ts	timestamp
VLC	València (= Valencia)
VM	virtual machine

An abstract finalization of the VLC pipeline

1. Object(ive)s

This project concerns environmental metrics provided to the public by the municipality of Valencia. Given that data points are published as (i) historic bulk updated with a substantial lag + (ii) live snapshot tables displaying only the latest readings, we have set out to achieve the following goals:

- load the historic data to TimescaleDB, an open-source solution specializing in time series
- create puller scripts (→ producers to Kafka) to emit each station's data when a timestamp advances
- build a pipeline that ingests updates, preserves history & provides queries + rollups
- follow the principles of reproducibility, security & understandability (Adkins et al., 2020, Chapter 6).

In the current context, 5 weather stations are monitored for data on air temperature, relative humidity, precipitation, barometric surface pressure, wind direction and wind speed as well as 11 air quality stations for NO2, O3, PM2.5, PM10 and SO2 readings (cf. Felici-Castell et al., 2023, p. 2).

2. Breakdown of implementation

Custom images have been built for air and weather producer scripts, which poll the VLC ODS API every 5 minutes, and send new data to Kafka topics (vlc.air/weather). These are consumed by JDBC connectors for **full-stack** and Python sinks for **lean-stack** deployment (cf. compose\compose_profiles.txt), forwarding messages to air/weather hypertables in TimescaleDB with time partitioning on column type timestamptz per best practice (Tiger Data, n.d.). The DB is then accessible through SQL. Example queries have been integrated into Grafana dashboards VLC Air Quality and VLC Weather. For data validation commands, check db/direct_sql_queries.txt.

Data safety: Hypertables are keyed on `(fiwareid, ts)` and the JDBC sink uses `insert.mode=upsert` on those PKs, so historical backfill + live ingestion can co-exist without duplicates.

To guarantee portability, we use Docker Compose, SQL init scripts and shell automation for configuring (i) connectors, (ii) Kafka, (iii) secrets handling and (iv) TimescaleDB. Our goal has been to ensure a reproducible infrastructure and application configuration without locking the stack to specific primitives. Core services have been set to `restart: unless-stopped` to auto-resume after exit events, and the configurators have all been created with idempotency in mind.

We have also created generic startup and installer scripts that detect the environment to enable flexible auto-start capabilities on kernel reboot (cf. scripts/vlc-start.sh, scripts/autostart/).

The data flow of the optional monitoring system treks on the route of JMX (metrics exposor) → JMX Exporter → Prometheus (scraper/converter) → Grafana (visualizer). Alertmanager has also been added to directly inform development about critical events by email.

3. Result

The data streaming system is capable of storing and processing incoming sensor data from the ODS API. With two docker networks—**internal** and **external**—cast up, security measures include (i) internal isolation for Kafka/Connect/DB, (ii) single ingress point via reverse proxy, and (iii) HTTP basic auth + secrets handling drills. The Kafka bootstrapper script creates two topics, one for air quality and another one for weather info. Also, CI/CD guidelines have been heeded to by setting up a gh Actions workflow (cf. `.github\workflows\test.yml`).

Per our view, the results achieved correspond to the original objectives to a surprisingly great extent. Historic chunks are loaded to the same DB that collects the live updates. The Python-based sidecars have been beefed up to offer robust fallback and shedding mechanisms when necessary (cf. `producer/resilience.py`). The pipeline elements do their prescribed dance together in a coherent fashion, showcasing that our decisions regarding the appropriate fittings were not completely random. Also, reliability and security principles were followed to a degree common sense allowed us to. With that said, it would be imprudent if we failed to mention a couple of catastrophies/moments of enlightenment encountered along the way, so please read on.

4. Reflection

In the development phase, we implemented and tested the pipeline in a local Ubuntu/ext4 environment. For P3, a virtual infrastructure has been provisioned declaratively via ARM using Bicep.

Virtually freezing

Symptom: FleetDiagnostics in Azure informed us about the machine failing to report status for VM agent. We enabled boot diagnostics and gathered information from the relevant logs.

Cause: The VM was being hammered by SSH brute-force attempts from many IPs → `sshd` hit `MaxStartups` throttling. This overwhelmed the VM, causing it to become unresponsive.

Measure: Tightening the NSG to only allow SSH from specific IPs to prevent future freezes from scanning attacks.

Virtually out of memory

Symptom: The VM booted with the full ~4 GiB RAM available. Later, Linux reported: “Out of memory: Killed process ... (java) ...”. The kernel invoked the OOM killer and shot a Java process down.

Cause: Small VM + several hungry services + no hard memory budget = kernel becomes the scheduler of system doom. One JVM getting to ~1.2 GiB RSS is enough to tip over the box.

Measure: Per budgetary constraints, we incorporated a **lean-stack** deployment option with reduced always-on JVM count (excluding profile `-ui` from `compose up` plus adding cheaper alternative sinks) and a `--skip-connect` flag for the Kafka bootstrapper. The tweaks solved the OOM issues.

BIBLIOGRAPHY

- Adkins, H., Beyer, B., Blankinship, P., Lewandowski, P., Oprea, A., & Stubblefield, A. (2020). *Building Secure and Reliable Systems: Best Practices for Designing, Implementing, and Maintaining Systems*. O'Reilly Media, Incorporated.
<http://ebookcentral.proquest.com/lib/badhonnef/detail.action?docID=6202784>
- Felici-Castell, S., Segura-Garcia, J., Perez-Solano, J. J., Fayos-Jordan, R., Soriano-Asensi, A., & Alcaraz-Calero, J. M. (2023). AI-IoT Low-Cost Pollution-Monitoring Sensor Network to Assist Citizens with Respiratory Problems. *Sensors*, 23(23), 9585.
<https://doi.org/10.3390/s23239585>
- Tiger Data. (n.d.). *Tiger Data Documentation | Hypertables*. Retrieved December 20, 2025, from <https://www.tigerdata.com/docs/use-timescale/latest/hypertables>

APPENDIX

Screenshots displaying milestones during finalization/optimization

```
systemctl status vlc.service
● vlc.service - VLC Data Pipeline Stack
   Loaded: loaded (/etc/systemd/system/vlc.service; enabled; preset: enabled)
   Active: active (exited) since Wed 2025-12-17 19:45:04 CET; 4s ago
     Process: 257337 ExecStart=/home/acidvuca/vlc/scripts/vlc-start.sh (code=exited, status=0/SUCCESS)
    Main PID: 257337 (code=exited, status=0/SUCCESS)
      CPU: 117ms

Dec 17 19:45:03 kulcs vlc-start.sh[257360]: Container kafka Waiting
Dec 17 19:45:03 kulcs vlc-start.sh[257360]: Container timescaledb Waiting
Dec 17 19:45:04 kulcs vlc-start.sh[257360]: Container kafka Healthy
Dec 17 19:45:04 kulcs vlc-start.sh[257360]: Container kafka Healthy
Dec 17 19:45:04 kulcs vlc-start.sh[257360]: Container timescaledb Healthy
Dec 17 19:45:04 kulcs vlc-start.sh[257360]: Container kafka Healthy
Dec 17 19:45:04 kulcs vlc-start.sh[257360]: Container timescaledb Healthy
Dec 17 19:45:04 kulcs vlc-start.sh[257360]: Container reverse-proxy Started
Dec 17 19:45:04 kulcs vlc-start.sh[257337]: [vlc-start] Stack started successfully
Dec 17 19:45:04 kulcs systemd[1]: Finished vlc.service - VLC Data Pipeline Stack.
```

Figure 2: Pipeline starting up via shell script launch

UI for Apache Kafka83b5a60 v0.7.2

Dashboard

vlc

Brokers

Topics

Consumers

Schema Registry

			02:21:54		
+	44	0	12/17/2025, 02:21:54	A10_OLIVERETA_60m 2025-12-17T01:00:00Z	{fiwareid:"A10_OLIVERETA_60m","ts":"2025-12-1
+	45	0	12/17/2025, 02:21:54	A02_BULEVARDSUD_60m 2025-12-17T01:00:...	{fiwareid:"A02_BULEVARDSUD_60m","ts":"2025-
+	46	0	12/17/2025, 02:21:54	A06_VIVERS_60m 2025-12-17T01:00:00Z	{fiwareid:"A06_VIVERS_60m","ts":"2025-12-17T0
+	47	0	12/17/2025, 02:21:54	A01_AVFRANCIA_60m 2025-12-17T01:00:00Z	{fiwareid:"A01_AVFRANCIA_60m","ts":"2025-12-
-	48	0	12/17/2025, 02:21:54	A03_MOLISOL_60m 2025-12-17T01:00:00Z	{fiwareid:"A03_MOLISOL_60m","ts":"2025-12-17

Key

Value

Headers

```
{
  "fiwareid": "A03_MOLISOL_60m",
  "ts": "2025-12-17T01:00:00Z",
  "so2": 0.0,
  "no2": 5.0,
  "o3": 54.0,
  "co": 0.0,
  "pm10": 17.0,
  "pm25": 10.0,
  "air_quality_summary": "Razonablemente Buena",
  "lat": 39.481112,
  "lon": -0.40881
}
```

Timestamp

12/17/2025, 02:21:54

Timestamp type: CREATE_TIME

Key Serde

String

Size: 36 Bytes

Value Serde

SchemaRegistry

Size: 202 Bytes

Figure 3: Topics getting data as seen through kafka-ui ✓


```
curl -I http://127.0.0.1:8080/

HTTP/1.1 302 Moved Temporarily
Server: nginx/1.27.5
Date: Thu, 18 Dec 2025 16:28:13 GMT
Content-Type: text/html
Content-Length: 145
Location: http://127.0.0.1:8080/grafana/
Connection: keep-alive
```

Figure 4: Grafana set to be reachable on the reverse proxy ✓

```
azureuser@vlc-vm /opt/vlc git:(main) (0.247s)
docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT max(ts) FROM weather.hyper;"

      max
-----
2025-12-22 15:00:00+00
(1 row)

azureuser@vlc-vm /opt/vlc git:(main) (0.255s)
docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT max(ts) FROM weather.hyper;"

      max
-----
2025-12-23 13:50:00+00
(1 row)
```

Figure 5: After a lot of tweaks, the VM chugs on nicely with the lean deployment setup ✓

```
azureuser@vlc-vm /opt/vlc git:(main) (0.158s)
docker compose -f compose/docker-compose.yml ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
compose-air-producer-1	vlc/air-producer:latest	"/usr/bin/tini -- py_..."	air-producer	8 days ago	Up 8 days
compose-air-sink-1	vlc/sink:latest	"python -u sink.py"	air-sink	8 days ago	Up 8 days
compose-kafka-1	confluentinc/cp-kafka:7.6.1	"/etc/confluent/docker..."	kafka	8 days ago	Up 8 days (healthy)
compose-timescaledb-1	timescale/timescaledb-ha:pg16-ts2.17-all	"/docker-entrypoint..."	timescaledb	8 days ago	Up 8 days (healthy)
compose-weather-producer-1	vlc/weather-producer:latest	"/usr/bin/tini -- py_..."	weather-producer	8 days ago	Up 8 days
compose-weather-sink-1	vlc/sink:latest	"python -u sink.py"	weather-sink	8 days ago	Up 8 days

Figure 6: Days in & days out, the pipes are all up fine ✓

```
~/vlc git:(main) 1 file changed, 11 insertions(+), 3 deletions(-) (2.984s)
docker compose -f compose/docker-compose.yml exec -T timescaledb psql -U vlc_dev -d vlc -c "SELECT * FROM (SELECT fwareid, 'air' AS data_type, MIN(ts) AS first_seen, MAX(ts) AS last_seen, COUNT(*) AS record_count, AVG(lat) AS lat, AVG(lon) AS lon FROM air.hyper GROUP BY fwareid) ALL SELECT fwareid, 'weather' AS data_type, MIN(ts) AS first_seen, MAX(ts) AS last_seen, COUNT(*) AS record_count, AVG(lat) AS lat, AVG(lon) AS lon FROM weather.hyper GROUP BY fwareid) t ORDER BY fwareid;"
```

fwareid	data_type	first_seen	last_seen	record_count	lat	lon
A01 AVFRANCIA_60m	air	2016-01-01 00:00:00+00	2025-12-28 18:00:00+00	61604	39.45782688794909	-0.34298623153225816
A02 BULEVARDSDUD_60m	air	2016-01-01 00:00:00+00	2025-12-28 18:00:00+00	61623	39.45039600548242	-0.39633756617849364
A03 MOLISOL_60m	air	2016-01-01 00:00:00+00	2025-12-28 18:00:00+00	52863	39.48111211036898	-0.40880899739826795
A04 PISTASILLA_60m	air	2016-01-01 00:00:00+00	2025-12-28 18:00:00+00	61623	39.458000953888406	-0.376643936841598
A05 POLITECNIC_60m	air	2016-01-01 00:00:00+00	2025-12-28 18:00:00+00	61622	39.47964449488079	-0.33740066192116475
A06 VIVERS_60m	air	2016-01-01 00:00:00+00	2025-12-28 18:00:00+00	61623	39.47964092511649	-0.36964822222842687
A07 VALENCIACENTRE_60m	air	2016-01-01 00:00:00+00	2025-12-28 18:00:00+00	37177	39.470547672521846	-0.3763976540446467
A08 DR_LLUCH_60m	air	2025-12-16 22:00:00+00	2025-12-28 18:00:00+00	255	39.466685000000005	-0.32828900000000002
A09 CABANVAL_60m	air	2025-12-16 22:00:00+00	2025-12-28 18:00:00+00	255	39.474391000000003	-0.32853499999999997
A10 OLIVERETA_60m	air	2022-01-01 00:00:00+00	2025-12-28 18:00:00+00	9915	39.46924422844211	-0.4059234329267481
A11 PATRATX_60m	air	2025-12-17 09:00:00+00	2025-12-28 18:00:00+00	197	39.459189000000004	-0.40141100000000003
W01 AVFRANCIA_10m	weather	2016-01-01 00:00:00+00	2025-12-28 18:10:00+00	52787	39.45781928937739	-0.34297814577595837
W02 NAZARET_10m	weather	2020-01-01 01:00:00+00	2025-12-28 18:10:00+00	26764	39.448530999736235	-0.333298000515326
W03 VALENCIAAEROPUERTO_10m	weather	2025-12-17 08:59:00+00	2025-12-28 16:50:00+00	179	39.489999999999991	-0.46999999999999994
W04 VALENCIAIADT_10m	weather	2016-01-01 00:00:00+00	2025-12-28 16:40:00+00	51636	39.479642190428024	-0.3696494630420954
W05 VALENCIA_UPV_10m	weather	2016-01-01 00:00:00+00	2025-12-19 18:30:00+00	51457	39.47964451765545	-0.3374008120662351

Figure 7: Backfill completed (with certain stations either being new or having unavailable historical data)

bucket_hour	temp_avg_c	humidity_avg	pressure_avg	wind_avg_ms	wind_avg_deg	precip_max_mm
2025-12-26 00:00:00+00	11.415384615384616	85.76923076923077	1007.8583333333332	1.4076923076923078	-24.878337007416974	0.3
2025-12-26 01:00:00+00	10.941666666666668	92.33333333333333	1008.2181818181817	1.0083333333333335	-40.160538132101976	0.3
2025-12-26 02:00:00+00	10.592857142857143	95.71428571428571	1008.0384615384615	0.9357142857142857	-56.51487631797287	0.4
2025-12-26 03:00:00+00	10.123076923076923	97.3076923076923	1007.85	1.0846153846153845	-63.30561457644386	0.3
2025-12-26 04:00:00+00	9.821428571428571	97.07142857142857	1007.5461538461539	1.4000000000000001	-66.198804834478	0.3
2025-12-26 05:00:00+00	9.87142857142857	95.71428571428571	1007.5538461538462	1.9999999999999998	-74.48388942295422	0.1
2025-12-26 06:00:00+00	10.053846153846154	93.53846153846153	1007.8666666666667	1.9846153846153844	-78.62806816240807	0
2025-12-26 07:00:00+00	10.423076923076923	87.38461538461539	1008.4333333333334	1.2692307692307692	-47.89622869338556	0
2025-12-26 08:00:00+00	10.884615384615383	85.38461538461539	1009	1.3615384615384616	-56.62164973517038	0
2025-12-26 09:00:00+00	11.2875	84.8125	1009.1	1.71875	-49.00203313516227	0
2025-12-26 10:00:00+00	12.014285714285714	83.64285714285714	1010.0384615384615	1.2142857142857146	-92.45721807500134	0
2025-12-26 11:00:00+00	12.10714285714286	84.21428571428571	1009.5461538461539	1.6928571428571426	-124.77251999477592	0
2025-12-26 12:00:00+00	14.578571428571427	74.92857142857143	1009.0538461538462	1.092857142857143	-118.09150519302491	0
2025-12-26 13:00:00+00	15.885714285714288	67	1009.0461538461539	1.3857142857142857	45.67553703735809	0
2025-12-26 14:00:00+00	15.726666666666667	68.53333333333333	1008.9928571428572	1.3933333333333333	63.32945493673982	0
2025-12-26 15:00:00+00	14.0625	77.6875	1009.6714285714286	2.1625	44.815156718014485	0
2025-12-26 16:00:00+00	13.608333333333334	81.16666666666667	1010.9166666666666	1.4749999999999999	27.89969823865789	0
2025-12-26 17:00:00+00	13.55	80	1011.5	0.85	20.000000000000004	0

(18 rows)

Figure 8: Hourly averages across weather stations for the given day

```

~/vlc git:(main) 4 files changed, 809 insertions(+), 70 deletions(-) (0.296s)
docker exec compose-prometheus-1 wget -qO- 'http://localhost:9090/api/v1/query?query=up' | jq
data: {
  "resultType": "vector",
  "result": [
    {
      "metric": {
        "__name__": "up",
        "instance": "connect:9406",
        "job": "connect"
      },
      "value": [
        1766865030.398,
        "1"
      ]
    },
    {
      "metric": {
        "__name__": "up",
        "instance": "localhost:9090",
        "job": "prometheus"
      },
      "value": [
        1766865030.398,
        "1"
      ]
    },
    {
      "metric": {
        "__name__": "up",
        "instance": "schema-registry:9405",
        "job": "schema-registry"
      },
      "value": [
        1766865030.398,
        "1"
      ]
    },
    {
      "metric": {
        "__name__": "up",
        "instance": "kafka:9404",
        "job": "kafka"
      },
      "value": [
        1766865030.398,
        "1"
      ]
    }
  ]
}

```

Figure 9: Prometheus vector result listing scrape targets up (= successfully scraped at given timestamp)

```

~/vlc git:(main) 4 files changed, 809 insertions(+), 70 deletions(-) (0.279s)
docker exec compose-prometheus-1 wget -qO- 'http://localhost:9090/api/v1/rules' | jq '[.data.groups[].rules[] | {state, name, query}]'
[
  {
    "state": "inactive",
    "name": "ConnectTaskFailed",
    "query": "connect_task_status == 0"
  },
  {
    "state": "inactive",
    "name": "ConnectRecordFailures",
    "query": "rate(connect_task_record_failures_total[5m]) > 0"
  },
  {
    "state": "inactive",
    "name": "ConnectDown",
    "query": "up{job=\"connect\"} == 0"
  },
  {
    "state": "inactive",
    "name": "KafkaOfflinePartitions",
    "query": "kafka_offline_partitions_count > 0"
  },
  {
    "state": "inactive",
    "name": "KafkaUnderReplicatedPartitions",
    "query": "kafka_under_replicated_partitions > 0"
  },
  {
    "state": "inactive",
    "name": "KafkaBrokerDown",
    "query": "up{job=\"kafka\"} == 0"
  },
  {
    "state": "inactive",
    "name": "NoAirDataIngested",
    "query": "increase(kafka_topic_messages_in_total{topic=\"vlc.air\"}[30m]) == 0"
  },
  {
    "state": "inactive",
    "name": "NoWeatherDataIngested",
    "query": "increase(kafka_topic_messages_in_total{topic=\"vlc.weather\"}[30m]) == 0"
  }
]

```

Figure 10: Prometheus showing alert conditions are not currently met, so no alert is firing. ✓

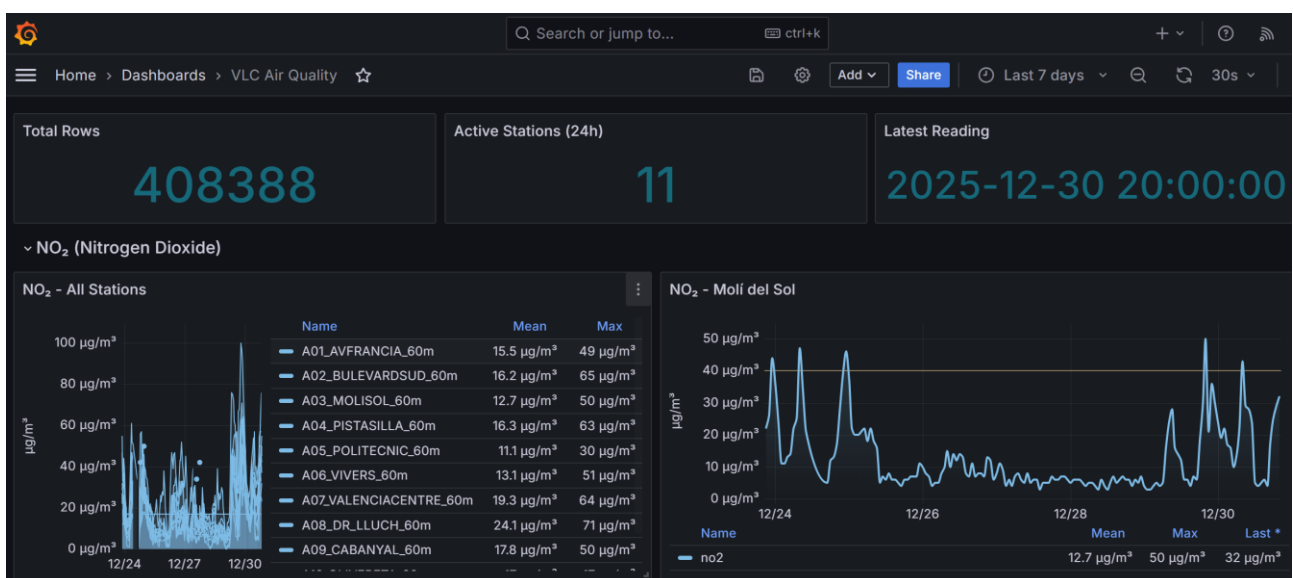


Figure 11: The first part of one of the topic-oriented Grafana dashboards set up